Prepare and import data

```
import pandas as pd
from transformers import T5Tokenizer, T5ForConditionalGeneration, Trainer, TrainingArguments


df = pd.read_csv('/content/domain_specific_chatbot_data.csv')
print(df.shape)
df.head()
```

```
(3000, 4)
```

	query	response	intent	domain
0	What are the side effects of the COVID-19 vacc...	Common side effects of the COVID-19 vaccine in...	side effects inquiry	healthcare
1	How can I schedule an appointment with my doctor?	You can schedule an appointment by calling our...	appointment booking	healthcare
2	What should I do if I miss a dose of my medica...	If you miss a dose, take it as soon as you rem...	medication inquiry	healthcare
3	How can I check my account balance?	You can check your balance by logging into you...	balance inquiry	finance
4	What is the interest rate for a personal loan?	The current interest rate for personal loans i...	loan inquiry	finance

Next steps: Generate code with df | View recommended plots | New interactive sheet

Clean data

```
df.isnull().sum()
```

	0
query	0
response	0
intent	0
domain	0

dtype: int64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   query     3000 non-null   object
 1   response  3000 non-null   object
 2   intent    3000 non-null   object
 3   domain    3000 non-null   object
dtypes: object(4)
memory usage: 93.9+ KB
```

Split data with only 2 values

```
from sklearn.model_selection import train_test_split
tokenizer = T5Tokenizer.from_pretrained('t5-base')

train_data, validation_data = train_test_split(df, test_size=0.2, random_state=42)
```

Reset index to match with TrainingArguments

```
train_data = train_data.reset_index(drop=True)
validation_data = validation_data.reset_index(drop=True)
```

```
train_data.head()
```

	query	response	intent	domain
0	What should I do if I miss a dose of my medica...	If you miss a dose, take it as soon as you rem...	medication inquiry	healthcare
1	What are the side effects of the COVID-19 vacc...	Common side effects of the COVID-19 vaccine in...	side effects inquiry	healthcare
2	What are the symptoms of flu?	Flu symptoms include fever, cough, sore throat...	flu symptoms inquiry	healthcare
3	How do I update my contact details on my account?	To update your contact details, log into your ...	contact update	finance
4	What are the side effects of the COVID-19 vacc...	Common side effects of the COVID-19 vaccine in...	side effects inquiry	healthcare

Next steps: Generate code with `train_data` | View recommended plots | New interactive sheet

Clean text then apply function to train and validation data

```
# Clean the text by removing unwanted characters
import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import string
import nltk
```

```
nltk.download('punkt_tab')
nltk.download('stopwords')


def cleaned_text(text):
    text = text.lower()
    word_tokens = word_tokenize(text)
    filtered_sentence = [word for word in word_tokens if not word in string.punctuation and word.isalnum()]
    return " ".join(filtered_sentence)


# Apply cleaning to dialogue and summary columns
train_data['query'] = train_data['query'].apply(cleaned_text)
train_data['response'] = train_data['response'].apply(cleaned_text)

validation_data['query'] = validation_data['query'].apply(cleaned_text)
validation_data['response'] = validation_data['response'].apply(cleaned_text)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Create the preprocessing function to get inputs, targets to tokenizer and assign input_ids from target to inputs labels. This action to make model compare the output from labels (bc labels is the base truth when model compare)

```
def preprocess_function(examples):
    inputs = tokenizer(examples["query"], padding="max_length", truncation=True, max_length=512)
    targets = tokenizer(examples["response"], padding="max_length", truncation=True, max_length=512)
```

```
    inputs["labels"] = targets["input_ids"]
    return inputs
```

Apply preprocessing function to get new train and val dataset

```
train_dataset = train_data.apply(preprocess_function, axis=1)
val_dataset = validation_data.apply(preprocess_function,  axis=1)
```

```
train_data['query'][0]
```

```
train_dataset[0]
```

```
{'input_ids': [125, 225, 3, 23, 103, 3, 99, 3, 23, 3041, 3, 9, 6742, 13, 82, 7757, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'attention_mask': [1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

```
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'labels': [3, 99, 25, 3041, 3, 9, 6742, 240, 34, 38, 1116,
38, 25, 1423, 3, 3227, 34, 966, 97, 21, 39, 416, 6742, 3, 99, 25, 3, 60, 3, 20305, 574, 39, 4640, 3175, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]}
```


```python
import torch

device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

Setup training model with model from T5ForConditionalGeneration pretrained. Then combine with Training Arguments as many parameters to help improve

performance of model.

```
# Model
model = T5ForConditionalGeneration.from_pretrained("t5-small")

training_args = TrainingArguments(
    output_dir="./results",          # Output directory for results
    evaluation_strategy="epoch",     # Evaluate once per epoch
    save_strategy="epoch",          # Save model at the end of each epoch to match evaluation strategy
    learning_rate=3e-5,              # Learning rate
    per_device_train_batch_size=8,  # Batch size for training
    per_device_eval_batch_size=8,   # Batch size for evaluation
    num_train_epochs=9,              # Increase number of epochs
    weight_decay=0.01,               # Strength of weight decay
    logging_dir="./logs",            # Directory for logging
    logging_steps=10,                # Log every 10 steps
    lr_scheduler_type="linear",      # Use linear learning rate scheduler with warmup
    warmup_steps=500,                # Number of warmup steps for learning rate scheduler
    load_best_model_at_end=True,     # Load the best model at the end of training
    metric_for_best_model="eval_loss", # Monitor eval loss to determine the best model
    save_total_limit=3,              # Limit the number of checkpoints to save
    # gradient_accumulation_steps= 2   # Simulate larger batch size if GPU memory is limite
)

trainer = Trainer(
    model = model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
)

trainer.train()
```

```
/usr/local/lib/python3.10/dist-packages/transformers/training_args.py:1575: FutureWarning: `evaluation_strategy` i
  warnings.warn(
```

[2700/2700 32:11, Epoch 9/9]

Epoch	Training Loss	Validation Loss
1	0.338100	0.173729
2	0.051200	0.034079
3	0.015200	0.005735
4	0.008100	0.002405
5	0.005500	0.001010
6	0.003600	0.000460
7	0.002800	0.000284
8	0.002100	0.000211
9	0.002000	0.000190

```
There were missing keys in the checkpoint model loaded: ['encoder.embed_tokens.weight', 'decoder.embed_tokens.weig
TrainOutput(global_step=2700, training_loss=0.7304270952871, metrics={'train_runtime': 1932.109,
'train_samples_per_second': 11.179, 'train_steps_per_second': 1.397, 'total_flos': 2923382911795200.0,
```

Save model and tokenizer

```
model.save_pretrained("/content/drive/MyDrive/Colab Notebooks/model_chatbox2")
tokenizer.save_pretrained("/content/drive/MyDrive/Colab Notebooks/model_chatbox2")
```

```
('/content/drive/MyDrive/Colab Notebooks/model_chatbox2/tokenizer_config.json',
 '/content/drive/MyDrive/Colab Notebooks/model_chatbox2/special_tokens_map.json',
```

```
    '/content/drive/MyDrive/Colab Notebooks/model_chatbox2/spiece.model',
    '/content/drive/MyDrive/Colab Notebooks/model_chatbox2/added_tokens.json')
```

Pull out the model and tokenizer for evaluation

```
model = T5ForConditionalGeneration.from_pretrained("/content/drive/MyDrive/Colab Notebooks/model_chatbox2").to(device)
tokenzier = T5Tokenizer.from_pretrained("/content/drive/MyDrive/Colab Notebooks/model_chatbox2")
```

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

```
'cuda'
```

Evaluation the chatbox by create function from Huggingface document. Then create the input section to test the chatbox

```
def chatbot(query):
    query = cleaned_text(query)
    input_ids = tokenizer(query,return_tensors="pt",max_length=250,truncation=True).to(device)

    # inputs = {key: value.to(device) for key, value in input_ids.items()}

    outputs = model.generate(
        input_ids["input_ids"],
        max_length=250,
        num_beams=5,
        early_stopping=True
    )
```

```
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    response = chatbot(user_input)
    print("Chatbot:", response)

# how to login to the system?
# where to find setting option?
# How can I schedule an appointment with my doctor?
# exit
```

```
You:  how to login to the system?
Chatbot: you can login to the system by visiting our website and filling out the application form
You: How can I schedule an appointment with my doctor?
Chatbot: you can schedule an appointment by calling our office or using our online portal
You: you can schedule an appointment by calling our office or using our online portal.
Chatbot: you can schedule an appointment by calling our office or using our online portal
You: where to find setting option?
Chatbot: you can find setting option by logging into the setting menu
You: exit
```